

Zadanie Ośrodek Narciarski

Ośrodek narciarski planuje przebudowę części tras i wyciągów i potrzebuje Twojej pomocy w zasymulowaniu ruchu narciarskiego po przebudowie.

Trasy i wyciągi

Ośrodek narciarski składa się z tras i wyciągów, którymi poruszają się narciarze i snowboardziści – zwani dalej łącznie sportowcami (w znaczeniu *osoby uprawiające sport* – niekoniecznie zawodowo).

Ośrodek narciarski możemy reprezentować jako graf skierowany. Krawędziami tego grafu są **trasy** (biegnące w dół stoku) i **wyciągi** (przevożące narciarzy w górę stoku). **Węzły** grafu wyznaczone są przez początki i końce tras (i są tylko tam – nie może być węzła w trakcie trasy). Każdy wyciąg ma dwie stacje – początkową i końcową. Stacje wyciągów znajdują się w węzłach. Trasy ani wyciągi nie mają punktów wspólnych – potencjalnie z wyjątkiem ich początków lub końców. Oznacza to, że sportowiec nie może zmienić trasy/wyciągu, którym się porusza, zanim nie dojedzie do końca tej trasy/wyciągu. Wyciągi (np. wyciąg krzesełkowy) mogą przechodzić ponad trasami czy innymi wyciągami.

Graf ośrodka jest silnie spójny – z każdego węzła można dojechać do dowolnego innego, poruszając się tylko trasami i wyciągami. W szczególności w każdym węźle początek ma przynajmniej jedna trasa lub wyciąg.

Każdy węzeł ma określoną **wysokość nad poziomem morza**. Do niektórych węzłów można dojechać samochodem lub autobusem (**węzły skomunikowane**) – to w takich węzłach sportowcy zaczynają i kończą dzień jazdy. Każdy sportowiec na koniec dnia wraca do tego samego węzła, w którym zaczął dzień.

Każda trasa ma określony **poziom trudności** (liczba całkowita od 0 do 10) i **czas przejazdu** (w sekundach). Dla uproszczenia przyjmujemy, że czas przejazdu daną trasą jest stały – nie zależy od poziomu zaawansowania sportowca ani od tego, czy sportowiec jeździ na nartach czy na desce.

Każdy wyciąg łączy dwa węzły o różnych wysokościach – w niższym jest stacja początkowa a w wyższym końcowa (można wjeżdżać tylko z dołu na górę). Przed stacją początkową ustawia się kolejka sportowców oczekujących na wjazd. Wyciąg co ustaloną liczbę sekund zabiera kolejną grupę pasażerów z kolejki i wiezie ją na górę. **Odstęp czasowy** między

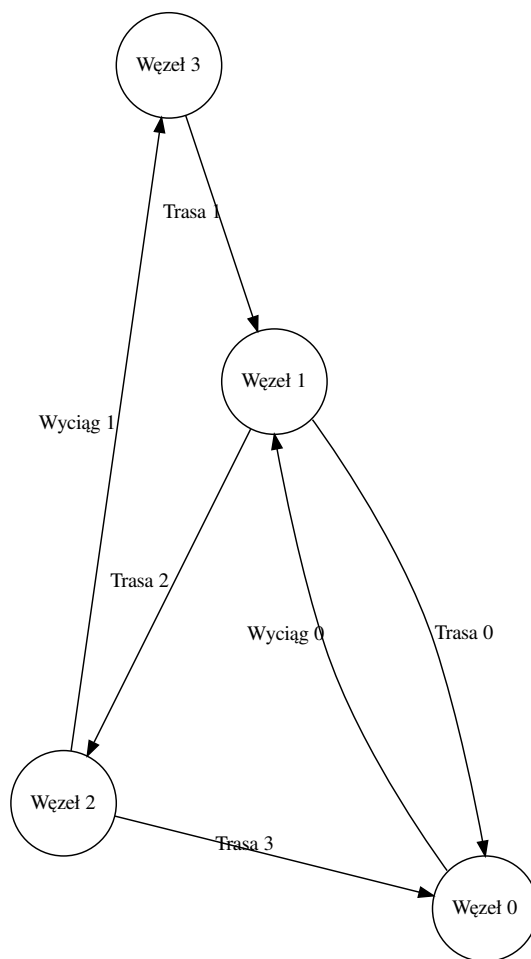


Figure 1: Przykładowy graf ośrodka narciarskiego (widok z przodu).

zabieraniem kolejnych grup pasażerów, **maksymalna wielkość grupy** (np. liczba siedzeń na pojedynczej kanapie wyciągu w przypadku wyciągu krzesełkowego) i **czas przejazdu** są stałe dla danego wyciągu. Z reguły wspomniany odstęp czasowy jest znacznie mniejszy niż czas przejazdu wyciągiem (np. wyciąg krzesełkowy ma wiele kanap i może wieźć w danym momencie wiele grup pasażerów).

Zakładamy, że wyciąg zabiera zawsze maksymalną liczbę pasażerów – puste miejsca mogą zdarzyć się tylko, jeśli w kolejce czeka za mało osób.

Sportowcy

Ogólne informacje

Z tras i wyciągów korzystają sportowcy. Każdy sportowiec przybywa na stok (do jednego z węzłów skomunikowanych) o określonej dla siebie godzinie z przedziału [9:00:00, 11:00:00) i rozpoczyna dzień jazdy (symulujemy tylko jeden dzień). W ciągu dnia sportowcy korzystają z dostępnych tras i wyciągów (kryteria sportowców co do wyboru kolejnych przejazdów opisane są dalej). Wszystkie wyciągi zaczynają pracę o 9:00:00, a kończą o 16:00:00.

O godzinie 15:00:00 każdy sportowiec zaczyna kierować się do punktu, w którym zaczął dzień jazdy – tak, aby zdążyć przed zamknięciem wyciągów. Powrotów do punktu startu nie symulujemy. Z punktu widzenia symulacji nie interesują nas zdarzenia dziejące się o godzinie 15:00:00 lub później. Wyjątkiem są wjazdy wyciągiem lub zjazdy trasą rozpoczęte niedługo przed 15:00:00. Symulacja powinna dokończyć je we właściwym dla nich czasie (nawet jeśli będzie to po 15:00:00) i poprawnie obsłużyć koniec tych aktywności (choćby pod kątem raportowania aktywności sportowców czy liczenia statystyk; oba aspekty opisane są dalej w treści zadania).

Można założyć, że przejazd dowolnym wyciągiem czy dowolną trasą trwa nie więcej niż godzinę. W szczególności przejazd zaczęty tuż przed 15:00:00 zakończy się na pewno przed 16:00:00. Nie daje to gwarancji, że każdy sportowiec da radę wrócić na czas do punktu startu z miejsca, w którym akurat się znalazł o godzinie 15:00:00, ale z perspektywy symulacji się tym nie przejmujemy.

Każdy sportowiec ma ustalony **poziom zaawansowania** określony liczbą całkowitą od 0 do 10.

Preferencje sportowców

Ogólne kryteria

Sportowcy oceniają atrakcyjność dostępnych tras, kierując się następującymi kryteriami:

- Dopasowaniem **trudności trasy** do umiejętności.
- Stopniem **zużycia trasy** (w ciągu dnia na trasie robią się muldy, które utrudniają jazdę).

W kolejnych wersjach symulacji zestaw kryteriów może się zmienić.

Dopasowanie poziomu trudności

Atrakcyjność d trasy wynikająca z dopasowania trudności trasy do poziomu sportowca zadana jest wzorem:

$$d(p_t, p_n) = \begin{cases} 0 & \text{dla } p_t \geq p_n + 5 \\ 1 - \frac{p_t - p_n}{5} & \text{dla } p_n + 5 > p_t \geq p_n \\ \max(0.2, 1 - \frac{p_n - p_t}{7}) & \text{dla } p_n > p_t \end{cases}$$

Gdzie:

- p_t – poziom trudności trasy,
- p_n – poziom zaawansowania sportowca.

Intuicje:

- $d(p_t, p_n) \in \langle 0, 1 \rangle$
- Jeśli $p_t = p_n$, to $d(p_t, p_n) = 1$.
- Gdy trasa jest za trudna, dopasowanie maleje liniowo do zera wraz z rosnącą rozbieżnością poziomów.
- Gdy trasa jest za łatwa, dopasowanie maleje liniowo (ale nieco wolniej) do wartości 0,2, wraz z rosnącą rozbieżnością poziomów.

Wyrównanie nawierzchni

Atrakcyjność w trasy wynikająca z wyrównania nawierzchni (gromadzące się w ciągu dnia nierówności utrudniając jazdę) zadana jest wzorem:

$$w(o_t, b_t, k_t) = b_t + (1 - b_t) \cdot o_t^{k_t}$$

Gdzie:

- $o_t \in (0, 1)$ – odporność trasy przeciw tworzeniu się nierówności.
- $b_t \in \langle 0, 1 \rangle$ – bazowa atrakcyjność trasy (zupełnie niewyrównanej).
- k_t – liczba wcześniejszych przejazdów trasą tego dnia, przez dowolnych sportowców (przejazdy na nartach i na desce liczą się tak samo).

Intuicje:

- $w(o_t, b_t, k_t) \in \langle 0, 1 \rangle$
- Dla $o_t \in (0, 1)$, wraz z kolejnymi przejazdami $w(o_t, b_t, k_t)$ maleje wykładniczo od 1 do b_t .
- Dla $o_t = 1$, $w(o_t, b_t, k_t) = 1$ niezależnie od liczby przejazdów.

Łączna atrakcyjność trasy

Wypadkowa atrakcyjność danej trasy dla sportowca jest ważoną średnią arytmetyczną atrakcyjności podanych wcześniej aspektów:

$$a(p_t, p_n, o_t, b_t, k_t) = \alpha_d \cdot d(p_t, p_n) + \alpha_w \cdot w(o_t, b_t, k_t)$$

Gdzie:

- $\alpha_d, \alpha_w \in \langle 0, 1 \rangle \wedge \alpha_d + \alpha_w = 1$ – wagi poszczególnych aspektów (indywidualne dla każdego sportowca).
- Pozostałe oznaczenia są opisane w poprzednich sekcjach.

Intuicje:

- $a(p_t, p_n, o_t, b_t, k_t) \in \langle 0, 1 \rangle$ – bo każda z funkcji składowych ma wartości z przedziału $\langle 0, 1 \rangle$, a średnia nie wychodzi poza $\langle \min, \max \rangle$ składowych.

Decyzje sportowców

Sportowiec w danym węźle może wybrać zjazd jedną z dostępnych tam tras lub wjazd jednym z dostępnych tam wyciągów.

Sportowiec rozważa wszystkie trasy wychodzące z danego węzła. Bierze też pod uwagę wszystkie wyciągi startujące z tego węzła i, dla każdego takiego wyciągu, rozważa trasy startujące z górnej stacji tego wyciągu. Spośród wszystkich rozważanych tras, sportowiec odszukuje tę najbardziej atrakcyjną (według kryteriów opisanych wcześniej; remisy rozstrzyga dowolnie). Jeśli znaleziona trasa wychodzi z obecnego węzła, to sportowiec po prostu nią zjeżdża. Jeśli znaleziona trasa wymaga wjechania wyciągiem, to sportowiec wybiera wjazd tym wyciągiem. Po wjechaniu wyciągiem sportowiec rozpoczyna procedurę wyboru od nowa. W szczególności może się zdarzyć, że sportowiec, zamiast zjechać upatrzoną wcześniej trasą (ze względu na którą wybrał wjazd wyciągiem), wybierze inną trasę (np. jeśli przez zjazdy innych osób trasa nieco się zużyła i inna trasa wypada teraz lepiej) lub wjazd kolejnym wyciągiem (jeśli jeszcze wyżej jest jeszcze bardziej atrakcyjna trasa).

Dla uproszczenia zakładamy, że sportowcy **mają pełną informację** co do aktualnych warunków na stoku (w szczególności wszystkich aspektów atrakcyjności wszystkich tras).

Czasami sportowcy dokonują spontanicznych wyborów. Przed analizą dostępnych tras i wyciągów, sportowiec z **prawdopodobieństwem $\varepsilon \in (0, 1)$ (stałym dla danego sportowca)** może pominąć całą procedurę wyboru opisaną wcześniej. W takim przypadku patrzy na wszystkie trasy i wyciągi zaczynające się w obecnym węźle i losuje jedną (lub jeden) z nich; każda trasa i wyciąg ma w takiej sytuacji równe prawdopodobieństwo wybrania.

Jak losować

Do generowania liczb pseudolosowych służy klasa `java.util.Random`.

Przykładowe użycie:

```
// Tworzenie generatora:
Random generator = new Random(); // lub new Random(ziarno);
// Losowanie int z przedziału [2, 6):
int losowyInt = generator.nextInt(2, 6);
// Losowanie double z przedziału [0, 1):
double losowyDouble = generator.nextDouble(0.0, 1.0);
```

W programie możemy:

- Albo utworzyć jeden generator na cały program i przekazać jego referencję wszystkim obiektom, które potrzebują losować.
- Albo utworzyć osobne generatory dla każdego obiektu, który potrzebuje losować. Jeśli korzystamy z konstruktora z ziarnem, należy zadbać o to, żeby każdy generator miał inne ziarno (w przeciwnym razie ciągi losowań mogą wyjść identyczne).

Generator tworzymy raz (lub raz na obiekt), a potem korzystamy z niego do wszystkich dalszych losowań. Błędem byłoby tworzenie nowych generatorów do każdego losowania z osobna.

Dokumentacja:

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Random.html>

(odsyłamy do dokumentacji Javy 21, a nie do najnowszej Javy 26, bo na komputerze students jest jeszcze Java 21)

Organizacja symulacji

Symulację przeprowadza się za pomocą **kolejki zdarzeń**. To z punktu widzenia programu linia czasu – lista zdarzeń posortowana niemalejąco ze względu na czas ich wystąpienia. Każdy obiekt wiedząc, że ma coś zrobić w przyszłości, wrzuca do kolejki stosowne zdarzenie ze sobą i z

czasem, kiedy zdarzenie ma nastąpić. Symulacja polega na pobieraniu z kolejki pierwszego (tj. o najniższym czasie wystąpienia) zdarzenia i proszeniu pobranego w nim obiektu o zareagowanie. Dzieje się tak przez zadany czas symulacji (nie: czas rzeczywisty) – od 9:00:00 do 14:59:59.

Czas w naszej symulacji mierzymy w wirtualnych (nie: rzeczywistych) sekundach.

Kolejkę zdarzeń można zaimplementować na różne sposoby. Możliwe realizacje to np.:

- Posortowana tablica pamiętająca, do którego miejsca jest wypełniona. Jej rozmiar powinien być podwajany w sytuacji, gdy zabraknie w niej miejsca na kolejne zdarzenia.
- Posortowana lista zdarzeń (lista w znaczeniu takim jak w pierwszym semestrze na C, tj. zestaw elementów, z których każdy pamięta swojego następnika).
- Kopiec (ang. heap) – pojawi się na II roku na ASD.

W tym zadaniu oczekujemy dowolnej z tych implementacji (wszystkie będą tak samo punktowane), dostarczających operacji wstawienia nowego zdarzenia do kolejki, pobrania pierwszego i sprawdzenia, czy kolejka jest pusta. Przy wstawianiu kilku zdarzeń o tym samym czasie zdarzenia później wstawione powinny być później wydawane z kolejki.

Wymagamy też, by w programie był **interfejs dla kolejki zdarzeń** (oczywiście implementowany przez Państwa kolejkę), tak by można było łatwo podmienić jedną implementację na inną (w tej części zadania wymagamy tylko jednej implementacji).

Próba pobrania zdarzenia z pustej kolejki powinna być wychwytywana asercją lub wyjątkiem.

Wskazówka: Zdarzenia mogą tworzyć hierarchię.

Oczywiście w bibliotece standardowej Javy są dostępne klasy realizujące kolejkę zdarzeń, ale w tym zadaniu (a przynajmniej w tej części) wymagamy Państwa własnej implementacji (jednej z podanych).

Kolejka oczekujących na wjazd

Sportowcy oczekujący na wjazd wyciągiem, ustawiają się w kolejkę.

Wymagamy implementacji tej kolejki w taki sposób, żeby operacje na kolejce były w czasie stałym (lub zamortyzowanym stałym). Przykładowe implementacje spełniające ten warunek:

- Bufor cykliczny, pamiętający początek i długość zapełnionego przedziału. Przy próbie dodania sportowca do pełnego bufora, należy utworzyć dwa razy większy. Bufor cykliczny był już omawiany na laboratorium w części grup – a w pozostałych grupach pojawi się na najbliższych zajęciach.
- Lista (jak w C). Każdy element pamięta poprzednika i następnika. Ponadto cała lista pamięta pierwszy i ostatni element.

W przypadku kolejki sportowców również nie należy korzystać z wbudowanych kontenerów Javy (innych niż tablice). Więcej informacji i wyjaśnienie tego ograniczenia jest w sekcji *Dodatkowe uwagi* na końcu treści zadania.

Statystyki

Na koniec symulacji dla każdej trasy i wyciągu należy wypisać na standardowe wyjście **łączną liczbę przejazdów** tą trasą czy wyciągiem.

Śledzenie sportowca

Ośrodek narciarski zainteresowany jest też indywidualnymi zachowaniami sportowców. Dla niektórych sportowców (odpowiednio oznaczonych na wejściu) symulacja powinna na bieżąco wypisywać na standardowe wyjście informacje o zdarzeniach z nimi związanymi. Interesuje nas, kiedy sportowiec zaczyna i kończy zjazd trasą. Należy odnotować też, kiedy sportowiec ustawia się w kolejce do danego wyciągu, zaczyna wjazd i schodzi z wyciągu.

Każde zdarzenie wypisujemy w osobnym wierszu. Każdy wiersz powinien zawierać na początku godzinę zdarzenia i dwukropek, a potem opis zdarzenia zawierający dane obiektów związanych ze zdarzeniem (np. numer sportowca, trasy czy wyciągu). Na przykład:

11:30:46: Sportowiec 56 rozpoczął zjazd trasą nr 7.

Wczytywanie danych symulacji

Jak wczytywać

Do wczytywania danych służy klasa `java.util.Scanner`. Jednym z sensownych podejść jest utworzenie jednego `Scannera` wczytującego kolejne linie wejścia i tworzenie osobnego `Scannera` dla każdej linii – interpretującego jej zawartość.

Przykładowe użycie:

```
// Tworzy scanner czytający standardowe wejście.
Scanner scannerWejscia = new Scanner(System.in);
// Sprawdza, czy została jeszcze jakaś linia do wczytania.
boolean czyZostalaLinia = scannerWejscia.hasNextLine();
// Wczytuje kolejną linię wejścia.
String linia = scannerWejscia.nextLine();

// Tworzy scanner czytający (i interpretujący) zawartość napisu `linia`.
Scanner scannerLinii = new Scanner(linia);
// Informuje scanner, że liczby zmiennopozycyjne używają kropki
// (zamiast przecinka) do oddzielania części ułamkowej (na studentsie
// domyślnie byłby przecinek).
scannerLinii.useLocale(Locale.ENGLISH);

// Dalsze metody do wczytywania:
int calkowita = scannerLinii.nextInt();
double rzeczywista = scannerLinii.nextDouble();
// słowo: ciąg znaków między odstępami (whitespace)
String slowo = scannerLinii.next();
// ... i naturalnie istnieją odpowiadające metody z prefiksem "has".
```

Można założyć, że podane na wejściu dane są poprawne – nie trzeba tego sprawdzać. W szczególności metody `Scannera` z prefiksem `has` raczej nie będą potrzebne (z wyjątkiem pojedynczych miejsc).

Dokumentacja:

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Scanner.html>

Opis danych

Ogólna struktura

Program powinien wczytać dane symulacji ze standardowego wejścia. Dane symulacji podzielone są na cztery sekcje – opis węzłów, opis tras, opis wyciągów i opis grup sportowców. Kolejne sekcje oddzielone są pustym wierszem. Każda sekcja zaczyna się linią zawierającą jedną liczbę całkowitą – liczbę obiektów (w znaczeniu ogólnym, nie Javowym) opisywanych w tej sekcji.

Schemat poglądowy:

```
liczba_węzłów  
<opisy_węzłów>
```

```
liczba_wyciągów  
<opisy_wyciągów>
```

```
liczba_tras  
<opisy_tras>
```

```
liczba_grup_sportowców  
<opisy_grup_sportowców>
```

Węzły, trasy, wyciągi i sportowców numerujemy kolejnymi liczbami całkowitymi od 0 zgodnie z ich kolejnością na wejściu. Numeracje różnych rodzajów obiektów są niezależne – w szczególności można spodziewać się istnienia naraz węzła nr 0 i trasy nr 0.

Węzły

Opis węzła składa się z jednej linii zawierającej jego wysokość bezwzględną (wyrażoną całkowitą liczbą metrów nad poziomem morza), parę współrzędnych całkowitych (x i y) oznaczających położenie węzła na mapie tras (przedstawiającej orientacyjny widok od przodu stoku; oś Y odpowiada wysokości) i, opcjonalnie, literę **s**, jeśli jest to jeden z węzłów skomunikowanych. Kolejne dane w linii oddzielane są pojedynczą spacją (można założyć to też w kolejnych sekcjach opisu danych wejściowych).

Informacje o położeniu węzłów na mapie można w pierwszej części zadania zignorować – będą one potrzebne do wizualizacji w drugiej części zadania.

Można założyć, że jest przynajmniej jeden węzeł skomunikowany.

Wyciągi

Opis wyciągu składa się z jednej linii zawierającej 5 liczb całkowitych: numer początkowego węzła, numer końcowego węzła, odstęp czasowy między kolejnymi grupami zabieranych pasażerów (w sekundach), maksymalna wielkość grupy pasażerów, czas przejazdu (w jedną stronę, w sekundach).

Trasy

Opis trasy składa się z jednej linii zawierającej 4 liczby całkowite i 2 liczby rzeczywiste, kolejno: numer początkowego węzła, numer końcowego węzła, poziom trudności, czas przejazdu (w sekundach), bazowa atrakcyjność niewyrównanej trasy i odporność trasy przeciw tworzeniu się nierówności.

Sportowcy

W symulacji może brać udział duża liczba sportowców, więc na wejściu definiujemy ich grupami. Wszyscy sportowcy w jednej grupie mają identyczne cechy – różnią się tylko numerem oraz, potencjalnie, godziną rozpoczęcia dnia na stoku.

Opis pojedynczej grupy składa się z trzech linii zawierających kolejno:

1. Liczbę sportowców w grupie (dodatnią), poziom zaawansowania, współczynnik spontaniczności *i*, opcjonalnie, literę *s*, jeśli sportowcy z grupy mają być śledzeni.
2. Wagi atrakcyjności poszczególnych aspektów tras, kolejno: dopasowania poziomem oraz wyrównania nawierzchni.
3. Numer startowego (skomunikowanego) węzła, godzinę rozpoczęcia dnia na stoku przez pierwszego sportowca z grupy (w formacie HH:MM:SS) oraz, jeśli jest ich więcej niż jeden, czasowy odstęp między początkami dnia kolejnych sportowców (jako całkowita nieujemna liczba sekund).

Sportowców również numerujemy kolejnymi liczbami całkowitymi od 0. W ramach każdej grupy sportowcy powinni być ponumerowani zgodnie z ich kolejnością pojawiania się na stoku. Numeracja sportowców między grupami powinna być zgodna z kolejnością pojawiania się grup w danych wejściowych.

Przykładowe dane

Plik z poniższymi danymi jest też na Moodle'u (kopiowanie z pliku PDF niestety pomija puste linie). Rysunek grafu tego ośrodka można obejrzeć na początku treści zadania.

```

4
859 5 1 s
919 3 6
879 1 2 s
1120 2 9

2
0 1 10 4 240
2 3 8 6 360

4
1 0 3 150 0.3 0.9998
3 1 7 180 0.3 0.9997
1 2 4 120 0.3 0.9998
2 0 2 60 0.3 0.99985

5
180 3 0.1
1 0
0 09:00:00 15
120 7 0.1
1 0
2 09:00:00 20
1 2 0 s
0.8 0.2
0 09:05:30
1 10 0 s

```



```
0.8 0.2
0 09:10:00
1 5 1 s
0.8 0.2
2 09:15:00
```

Dodatkowe uwagi

- Prosimy pamiętać, że to zadanie z programowania obiektowego, zdecydowanie nie należy się obawiać korzystania z klas, obiektów, konstruktorów, zakresów widoczności itp. Np. idealnie działający program z jedną tylko klasą będzie odczytany jako nieograniczona sympatia dla okrągłych liczb parzystych mających nieskończenie wiele dzielników.
- To większe zadanie, prosimy o rozwiązywanie go w sposób przyrostowy. Przykładową sensowną kolejnością jest zaczęcie od konstrukcji grafu tras i wyciągów. Później można zaimplementować kolejkę zdarzeń i uruchomić symulację z pojedynczym sportowcem (podejmującym jeszcze przypadkowe decyzje) i uproszczonym zachowaniem wyciągów (z jednoosobową kolejką oczekujących). Dalej można rozbudowywać kolejne funkcjonalności aż do uzyskania pełnego rozwiązania. Oczywiście nie wymagamy konkretnej kolejności pracy nad rozwiązaniem (oceniamy ostateczną wersję), ale wyznaczenie sobie pośrednich celów, po których sprawdzamy poprawność tego, co już zaimplementowaliśmy, pomaga w sprawnym ukończeniu rozwiązania.
- Kod większych programów wymaga refaktoryzacji. Często dopiero po napisaniu działającego programu zauważa się możliwe usprawnienia kodu, poprawiające czytelność i jakość kodu (typowy przykład – po napisaniu rozwiązania zauważa się, że dwie metody mają podobny fragment kodu, który można, a nawet należy, wydzielić w postaci osobnej metody). Ta faza tworzenia oprogramowania wymaga czasu. Prosimy więc nie zostawiać pisania programów na ostatnią chwilę.
- Zadanie zostało tak zaprojektowane, żeby dało się je dość łatwo zrobić, korzystając z tablic. Ponieważ nie znamy jeszcze kontenerów ze standardowej biblioteki Javy, nie możemy ich w tym zadaniu wymagać. A ponieważ wszyscy powinni mieć takie samo zadanie do zrobienia, to **nie wolno** w tym zadaniu korzystać z kontenerów innych niż tablice (własnoręcznie napisane struktury danych są w porządku).
- Kod rozwiązania powinien być podzielony na pakiety – grupujące klasy, które realizują podobną funkcjonalność.
- Rozwiązanie ma działać na komputerze students. Students ma Javę w wersji 21 – więc takiej też wersji warto używać, pracując nad zadaniem.
- Java domyślnie nie sprawdza asercji. Przy pracy nad rozwiązaniem warto mieć je włączone (przy ocenianiu rozwiązań będą włączone). Asercje włącza się argumentem `-ea` polecenia `java` (np. w IntelliJ ten argument można ustawić tutaj: **Edit Configurations > Modify options > Add VM options**).
- Drugie duże zadanie będzie kontynuacją pierwszego. Zadbanie o przejrzystą strukturę programu i dobrą jakość kodu w pierwszym zadaniu (pomocze uzyskać dobrą ocenę, wiadomo, ale także) ułatwi Państwu dalsze rozwijanie symulacji w zadaniu drugim.